Ders 12 Genişletilmiş Notu: Bağlantı Yönetimi, Akış Denetimi ve ARQ Yöntemleri

Ders 12 Genişletilmiş Notu: Bağlantı Yönetimi, Akış Denetimi ve ARQ Yöntemleri

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Veri Bağlantı Katmanının Görevleri
- 2. Bağlantı Yönetimi
- 3. Akış Denetimi: Hangi Katman, Hangi Sorun?
- 4. Dur ve Bekle Akış Denetimi
- 5. Kayan Pencere Akış Denetimi
- 6. Hata Denetimi ve ARQ
- 7. Go-Back-N
- 8. Sırtlama (Piggybacking)
- 9. Seçici Ret / Seçici Yineleme (Selective Reject / Selective Repeat)

1. Veri Bağlantı Katmanının Görevleri

Hoca derse dönemin 7. haftasında olduğumuzu hatırlatarak başladı. Bu dersin konusu bağlantı yönetimi ve akış denetimiydi. Önce veri bağlantı katmanının (data link layer) ya da bağlantı denetiminin (link control) neler yapması gerektiğini topladı:

- **Çerçeveleme (framing):** Bunu daha önce konuşmuştuk. Zamanlamanın sağlanması için çerçevelerin başlangıç ve bitiş noktalarının belirlenmesi gerekir.
- **Akış denetimi (flow control):** Gönderici ile alıcı arasındaki veri akışını takip etmek ve alıcının alabileceğinden fazla bilgiye maruz kalmasını engellemek. Bu dersin ana konularından biri.
- **Hata denetimi (error control):** Hat üzerinde her zaman gönderim hataları oluşabilir. Bunların denetlenmesi gerekir.
- **Adresleme (addressing):** Bu seviyede de mutlaka bir adresleme olmalıdır.
- **Bağlantı yönetimi (link management):** Bağlantının kurulması, sürekliliğinin sağlanması ve bağlantı üzerinden kimin ne zaman bilgi göndereceğinin belirlenmesi.

1.1 Alıcı Neden Boğulur?

"Alıcının alabileceğinden fazla bilgiye maruz kalması" ne demektir? Hoca normal iletimi şöyle anlattı: Göndericinin bir tamponu (buffer) vardır. Bilgiyi oraya koyar, alttaki katmana verir. Karşı tarafta alıcı bilgiyi alttan alır, kendi tamponuna yerleştirir, üzerinde işlem yapar ve devam eder. Bu düzende alıcıda yığılma oluşmasının birkaç nedeni vardır:

- **Farklı hızlar:** Hattın yapısı gereği iki uç farklı hızlarda çalışıyor olabilir. Karşı tarafın gönderme hızı ile sizin alma hızınız farklı olabilir.
- **Simetrik olmayan iş yükü:** Hem gönderici hem alıcı çerçeveler üzerinde işlem yapar. Bu işlemlerin simetrik olması gerekmez. Göndericinin gönderirken yaptığı iş az, alıcının alırken yaptığı iş daha yoğun olabilir. Bu da alıcı tarafında yığılmaya yol açar.

- **Farklı kaynaklar:** İki sistem hiçbir zaman denk olmak zorunda değildir. Biri CPU zamanı ya da RAM bakımından daha zayıf olabilir. Zayıf olan alıcıysa yine birikme olur.

Tampon bu birikmenin kontrollü olmasını sağlamak için vardır. Fakat tampon her zaman sınırlıdır. Tampon dolduktan sonra çerçeveler düşürülür (drop). Bunu istemeyiz, çünkü hattın verimli kullanılmasını (utilization) bekleriz. Mümkünse bir bilgiyi tek seferde göndermek isteriz. Her yeniden gönderimde hattaki kapasiteyi boşa harcarız.

2. Bağlantı Yönetimi

Bağlantı yönetiminin (line discipline) temel sorusu şudur: Hat hangi anda ve kim tarafından kullanılabilir? Bu sorunun cevabı kullanılan ortamın kısıtlarına bağlıdır. Hoca iki hat yapısını hatırlattı:

- **Noktadan noktaya (point-to-point):** Özellikle geniş alan ağlarında (WAN) kullanılır. Bütün uç noktalara ayrı ayrı hat çekmek yerine uç noktalardaki hatlar, geçen hafta konuştuğumuz çoğullama (multiplexing) ile birleştirilir ve tek bir hat üzerinden aktarılır. Nedeni maliyettir.
- **Çok noktalı (multipoint):** Yerel ağlarda (LAN) daha yaygındır. Ortak paylaşılan bir alan vardır. Kullanılan teknolojiye göre (Bluetooth, Wi-Fi, RF seviyesinde başka teknolojiler) spektrumun bir bölümü kullanılır. Bir istasyon ortamı kullanırken bir başkası da kullanmaya kalkarsa çarpışma (collision) olur ve ortam kullanılamaz hale gelir.

Bu iki farklı yapının her biri için hattın yönetimini sağlayan bir yöntem geliştirilmiştir: Sorgu/Onay (ENQ/ACK, Enquiry/Acknowledgement) ve Yoklama/Seçme (Poll/Select).

2.1 ENQ/ACK: Noktadan Noktaya Hatlar

ENQ/ACK daha çok noktadan noktaya ağlarda kullanılır. Burada haberleşmede birbirine eşit düzeyde iki taraf vardır. Hattı kullanabilecek zaten iki olası uç vardır. Taraflardan biri iletişimi başlatmak için harekete geçer. Diğeri olumlu cevap verirse iletişim olur, olumsuz cevap verirse iletişim başlatılamaz.

İletişim başlatılamazsa ne olur? Bir taraf konuşmak istiyor, diğeri müsait değil diyor. Genelde birkaç kez yeniden denenir. Her seferinde olumsuz cevap gelirse bir geri çekilme (back-off) mekanizması devreye girer. Bir süre beklenir ve karşı tarafın rahatlaması sağlanır.

Bu mekanizmada ENQ (enquiry) mesajı, karşı tarafa haberleşmek istediğimizi bildirmek için kullanılır. Karşılığında olumlu bir cevap (ACK, acknowledgement) ya da olumsuz bir cevap (NAK, negative acknowledgement) alınır.

> **Hocanın vurgusu:** Hata yalnız veride olmaz, kontrol bilgisinde de mutlaka hata oluşabilir. Gönderdiğiniz ENQ'ya olumlu ya da olumsuz hiçbir cevap alamazsanız iki ihtimal vardır: Ya sizin gönderdiğiniz bilgi gitmemiştir ya da karşı taraf bilgiyi alıp cevap üretmiş, ama cevap yolda kaybolmuştur. Bu iki durumu birbirinden ayırt etme şansınız yoktur. "Bunu lütfen unutmayın."

Çözüm şudur: Belli bir süre sonra hâlâ cevap alınamıyorsa bilgi yeniden gönderilir. Denemekten çekinilmez.

ENQ/ACK akışının görsel hali üç aşamadan oluşur:

1. **Bağlantı kurulumu (connection setup):** Gönderici ENQ yollar, karşı taraftan cevap alır.
2. **Veri aktarımı:** Bilgi gönderilir, her çerçeve için ACK alınır.
3. **Bağlantının sonlandırılması:** Gönderici EOT (End of Transmission) mesajını yollar.

> **Ders sonrası not:** Hoca kurulum aşamasını anlatırken "ona cevap aldığınız bir Enquiry var" ve veri aşamasında "karşılığında Enquiry almak" diyor. ENQ'ya ve veri çerçevelerine verilen cevap ENQ değil, ACK'dır.

2.2 Poll/Select: Çok Noktalı Hatlar

Poll/Select farklı bir yapı için kullanılır: çok noktalı bir bağlantı, yani bir veri yolu (bus). Bu yola birden fazla cihaz bağlıdır. Bağlantının yönetimi için bir birincil (primary) istasyon olması

beklenir. Diğer cihazlara ikincil (secondary) denir. Hattın düzgün yönetilmesi için yönetimin tek bir istasyon tarafından yapılması gerekir. Bu istasyon, adı üstünde, birincildir.

> **Hocanın vurgusu:** İletişimi her zaman yöneten ve yönlendiren birincildir. Bunu unutmayın.

İki durum vardır:

- **Select:** Birincilin doğrudan konuşmak istediği bir ikincil varsa, birincil ona bir SEL (select) çerçevesi yollayarak iletişime geçmek istediğini söyler.
- **Poll:** İkinciller iletişim başlatmak, birincile veri aktarmak istiyorsa birincilin sorgularına cevap vermelidir. Bu sorgu POLL komutudur. Birincil, çoğu zaman sırayla (round-robin benzeri), ikincillere tek tek gönderecek bilgileri olup olmadığını sorar. Gönderecek verisi olan ikincil doğrudan verisini aktararak cevap verir.

Burada akla bir soru gelebilir: Bir veri yolu var ve POLL gönderildi. Herkes aynı anda cevap verir ve ortalık karışır mı? Hayır. Çünkü hattaki birincil dışındaki bütün cihazlar ikincildir ve hepsi aynı hattı dinler. İkincilleri birbirinden ayırt eden bir adresleme mekanizması gerekir. Dolayısıyla POLL isteği herkese yayın (broadcast) olarak gönderilmez. Her ikincile ayrı bir POLL sorgusu gönderilir. Elinde verisi olan, verisiyle cevap verir. Verisi yoksa NAK göndererek cevap verir.

Select tarafında birincil, SEL çerçevesinin içine ikincilin adresini koyar. İkincil haberleşebilecek durumdaysa ACK, değilse NAK ile cevap verir.

Akışlar şöyledir:

- **Select akışı:** SEL → ACK → veri → ACK → veri → ACK → … → EOT. Haberleşmeyi başlatan birincildir. Bitime de birincil, ikincile EOT göndererek karar verir.
- **Poll akışı:** POLL → (verisi varsa) veri → ACK → … → EOT. Yapı simetriktir. Bu kez veriyi gönderen ikincildir ve bitirmek istediğinde EOT çerçevesini o gönderir. Verisi yoksa POLL'a NAK ile cevap verir.

Fakat hattın kontrolü her durumda birincildedir. İletişimin ne zaman başlayacağına, ne zaman biteceğine ve hangi ikincile söz hakkı verileceğine birincil karar verir. İkincil veri aktarımına başladıktan sonra, belli bir süre sonunda birincilin ikincile bir zaman aşımı (timeout) çerçevesi gönderip iletişimi sonlandırması da mümkündür.

3. Akış Denetimi: Hangi Katman, Hangi Sorun?

Akış denetimi ağda farklı katmanlarda farklı şekillerde gerçeklenir. Elimizdeki problem, bilginin bir noktadan diğerine gönderilmesidir.

- **Tek atlama (single hop):** İki nokta birbirinden tek atlama uzaklıktaysa akış denetimiyle veri bağlantı katmanı ilgilenir. Uçlar birbirine oldukça yakındır. Akışın bozulmasının temel nedeni alıcının tıkanması, kapasitesi ya da işleme sorunudur. Yapılması gereken, göndericiyi alıcının hızına indirmek, alıcının boğulmasını engelleyecek bir seviyeye getirmektir.
- **Çok atlama:** Araya "uçurumlar, dağlar", çok sayıda atlama ve başka ağlar girdiyse bu işle taşıma katmanı (transport layer) ilgilenir. Orada da akış denetimi yapılır, ama nedeni ve yöntemi farklıdır. Paketler bir ağdan geçer, başka bir ağa, oradan başka bir ağa gider. Burada sorunu çıkaran hedef bilgisayarın kapasitesi değil, aradaki ağların kapasitesidir. O ağlardaki yoğunluk ya da bir sorun akışımızı bozabilir.

Bu dersin konusu, veri bağlantı katmanında, aynı ağ içindeki cihazlar arasındaki akış denetimidir. İki yöntem konuşuldu: dur ve bekle (stop-and-wait) ve kayan pencere (sliding window).

4. Dur ve Bekle Akış Denetimi

4.1 Günlük Hayattan Bir Örnek

En temel yöntem, günlük hayatta uyguladığımız bir yapıdır. Hocanın örneği: Markette kasadasınız. Kasiyer "kartınız var mı?" diye soruyor. Kartınız evde kalmış, "cep telefonu numaranız olur" diyor. Numarayı söylemeye başlıyorsunuz: "0532 277" diyorsunuz ve duruyorsunuz. Kasiyer başıyla bir işaret yapıyor ya da yazdığını görüyorsunuz. "67" diyorsunuz, bir işaret daha.

Sonra “88” diyorsunuz ve devam ediyorsunuz. Yani bir bilgiyi söylüyor, karşı tarafın onu işlemesini bekliyor, sonra devam ediyorsunuz. Dur ve beklenin en temel hali budur.

Fakat bu yaklaşım veri aktarım hızını sınırlar. Kapasiteyi etkin kullanmak için farklı teknikler kullanılır. Bunların en bilineni ve günümüzde de bolca kullanılanı kayan penceredir.

4.2 Çalışma Mantığı

Dur ve beklede yöntemin adının söylediği yapılır: Gönderilen her çerçeveye karşılık bir geri bildirim beklenir. Elinizdeki veriyi gönderirsiniz. Bir sonraki parçayı göndermek için ilk gönderdiğinize karşılık bir cevap beklersiniz.

ACK beklemenin mantığı nedir? Bir bilgi gönderdiniz. Karşıya gidip gitmediğini bilmiyorsunuz. Bununla ilgili bir geri bildirime ihtiyacınız var. ACK bu yüzden vardır.

> **Hocanın vurgusu:** Bir bilgi gönderdiniz ve size hiçbir şey geri gelmiyor. Neden gelmediğini bilmiyorsunuz: Bilgi karşıya mı gitmedi, yoksa karşılığında gönderilen ACK mı size ulaşmadı? İkisi de olabilir. Bunu ayırt etme şansınız yok. “Bunu lütfen unutmayın.”

Dur ve beklede akış denetimi şöyle sağlanır: Alıcı kendisine gönderilen çerçeveler yüzünden sıkıntı yaşıyorsa, yani tamponu dolmaya başladıysa ya da çerçeve işleme hızı göndericinin hızına yetişemiyorsa, ACK göndermeyi yavaşlatır. Her çerçeveye hemen “tamam, tamam, tamam” demek yerine biraz bekler, “tamam” der, biraz daha bekler, “tamam” der. Bu ufak bekletmelerle göndericiyi kendi hızına çeker.

4.3 Hat Kullanım Oranı ve a Parametresi

Dur ve beklenin sorunu şudur: Bir veri hızınız (bant genişliği) var, ama onu doğrudan etkin kullanmıyorsunuz. Bir çerçevenin gönderilip onaylanması için geçen süre şu parçalardan oluşur:

- Çerçevenin iletim süresi t_{frame},
- yayılma gecikmesi (propagation delay) t_{prop},
- karşı tarafın çerçeveyi işleyip bir ACK üretmesi ve ACK’nın iletim süresi t_{ack},
- ACK’nın geri gelirken yayılma gecikmesi, yine t_{prop}.

$$T = t_{frame} + t_{prop} + t_{ack} + t_{prop}$$

Bu sürenin tamamında bir çerçeve göndermiş ve karşının aldığından emin olmuş olursunuz. Yani çerçeveyi iletmek için gereken süreden fazlasını, çerçevenin iletildiğini anlamak için harcarsınız.

ACK ve çerçevenin ayrı ayrı yazılmasının nedeni şudur: ACK, gönderilen veri çerçevesinden çok daha küçüktür. Bu yüzden t_{ack} çoğu zaman ihmal edilebilir. Dört terimden biri atılınca toplam süre şuna iner:

$$T \approx t_{frame} + 2\,t_{prop}$$

Hat kullanım oranı (utilization), çerçeveyi göndermek için harcanan sürenin toplam süreye oranıdır. Hat kapasitesinin ne kadarını kullanabildiğinizi gösterir:

$$U = \frac{t_{frame}}{t_{frame} + 2\,t_{prop}}$$

Yayılma gecikmesinin çerçeve iletim süresine oranına a denir:

$$a = \frac{t_{prop}}{t_{frame}}$$

Pay ve payda t_{frame}'e bölünürse (ara adım eklendi) formül şu hale gelir:

$$U = \frac{1}{1 + 2a}$$

Yayılma gecikmesi mesafenin yayılma hızına oranıdır: $t_{prop} = d/v$. Çerçeve iletim süresi ise çerçeve boyunun veri hızına (bant genişliğine) oranıdır: $t_{frame} = L/R$. Böylece:

$$a = \frac{d/v}{L/R}$$

4.4 Örnek: 1000 km'lik Hatta Dur ve Bekle

Verilen:

- İki nokta arası mesafe $d = 1000$ km $= 10^6$ m.
- Veri hızı $R = 155{,}52$ Mbps.
- Yayılma hızı $v = 2 \times 10^8$ m/s ("200 bin km/s"). Hoca bu kadar hızlı olduğuna göre herhalde optik bir hattan bahsedildiğini söyledi.
- Çerçeve boyu $L = 424$ bit.
- **Soru:** Dur ve bekle kullanılırsa hattın kullanım oranı nedir?

Hoca öğrencilere hesaplamak isteyen olup olmadığını sordu, sonra yolu gösterdi: Önce t_{prop} mesafe / hızdan, t_{frame} çerçeve boyu / veri hızından bulunur. Sonra a hesaplanır ve $U = 1/(1 + 2a)$'ya konur.

Çözüm:

$$t_{prop} = \frac{10^6}{2 \times 10^8} = 5 \times 10^{-3} \text{ s} = 5 \text{ ms}$$

$$t_{frame} = \frac{424}{155{,}52 \times 10^6} \approx 2{,}726 \times 10^{-6} \text{ s} \approx 2{,}726 \ \mu\text{s}$$

$$a = \frac{5 \times 10^{-3}}{2{,}726 \times 10^{-6}} \approx 1834$$

$$U = \frac{1}{1 + 2 \cdot 1834} \approx \frac{1}{3669} \approx 2{,}73 \times 10^{-4}$$

Yani hat zamanının yalnızca on binde 2,7 kadarı kullanılıyor. Sayılar Python ile doğrulandı.

> **Ders sonrası not:** Hoca veri hızını okurken "55.52 Mbps" diyor. Kısa nottaki slayt değeri 155,52 Mbps'tir; yukarıdaki çözüm bu değerle yapıldı.
>
> **Ek bilgi (derste anlatılmadı):** 155,52 Mbps büyük olasılıkla SONET OC-3 / SDH STM-1 hızıdır. 424 bit ise 53 baytlık ATM hücresinin boyudur. Örnek büyük olasılıkla optik hat üzerinden ATM hücresi taşıyan bir bağlantıyı modelliyor.

Hoca sonucu şöyle yorumladı: Dur ve beklede her seferinde karşı taraftan bir bilginin dönmesini beklemek gerekir. Bu yüzden hat kullanım oranı çok düşüktür. Kısa mesafelerde bunu pek fark etmezsiniz, çünkü iletim hızınız çok yüksektir. Ama bin kilometre gibi bir mesafede gecikmeler çok daha yüksektir. Gecikme arttıkça bir çerçevenin döngü süresi artar, birim zamanda aktarılan veri azalır ve hat etkin kullanılamaz.

5. Kayan Pencere Akış Denetimi

5.1 Temel Fikir

Dur ve beklede bir çerçeve gönderilir ve cevap gelene kadar başka hiçbir şey gönderilemez. Aradaki zaman boşa gider. Kayan pencerede bunun yerine bir pencere boyutu belirlenir ve pencere içindeki çerçeveler art arda ("tak tak tak") gönderilir. Böylece her çerçeveyi tek tek beklemek yerine bekleme süresi kısaltılır.

Bunun için gerekenler:

- **Tampon:** Her iki tarafta da bir pencere oluşturulacağı için hem göndericide hem alıcıda pencere kadar bilgi saklayabilecek bir tampon alanı gerekir.
- **Numaralandırma:** Pencere içinde çok sayıda çerçeve gönderileceği için çerçeveler birbirinden ayırt edilebilmelidir. Her çerçeveye bir numara (ID) verilir.
- **Numara alanı:** Çerçeve numarası alanı n bit ise 2^n farklı numara ifade edilebilir. Numaralar 0'dan başlar, $2^n - 1$'e kadar gider. Pencere kaydıkça numaralar başa döner, yani numaralar modüler aritmetikle hesaplanır.

  Ders sonrası not: Hoca numaraların "n'e göre modülü alınarak" hesaplandığını ve "n eksi bire kadar" gidildiğini söylüyor. n bitlik numara alanında numaralar $0 \dots 2^n - 1$ arasıdır ve işlem mod 2^n ile yapılır. Hoca örnekte bunu doğru kullanıyor: 3 bit ile 0–7.

5.2 ACK Numarası

Her çerçeve gönderildiğinde karşı taraftan bir geri bildirim beklenir. Fakat bu geri bildirim, alınan çerçevenin numarasını değil, beklenen bir sonraki çerçevenin numarasını taşır. Alıcı 0 numaralı çerçeveyi aldıysa ACK 1 gönderir. Böylece bir sonraki çerçeve olarak 1'i beklediğini belirtir.

Pencere büyüklüğü, numara sayısı 2^n ise bunun bir eksiği, $2^n - 1$ olarak seçilir. Nedeni aşağıda (5.5) açıklandı.

Örnek: Çerçeve numaraları için 3 bit kullanılsın. $2^3 = 8$ farklı çerçeve ifade edilebilir: 0, 1, 2, 3, 4, 5, 6, 7. Sonraki tur yine 0'dan başlar. Pencere boyutu 7'dir.

5.3 Gönderici ve Alıcı Pencereleri

Gönderici tarafı: Pencere en baştan, 0'ı gösterecek şekilde ayarlanır. Bilgi gönderildikçe bir işaretçi (ok) bir sonrakine geçer. Yani her gönderilen çerçeveden sonra işaretçi sağa kayar. İşaretçinin sağındakiler gönderilmeyi bekleyen, solundakiler gönderilmiş çerçevelerdir.

> **Hocanın vurgusu:** Dikkat edin, gönderilmiş olması ACK alındığı anlamına gelmiyor.

ACK alındığında ne olur? Karşı taraftan ACK geldiyse artık o çerçevelerle işimiz kalmamıştır. Onları gönderdik, karşı taraf sağlam aldı ve ACK gönderdi. Her şey yolunda olduğuna göre yeni çerçeveler göndermeye başlayabiliriz: Pencere sağa kaydırılır.

Alıcı tarafı: Kayan pencere kullanılıyorsa iki taraf pencere boyutu konusunda anlaşmış olmalıdır. Alıcının tamponu da benzer şekilde düzenlenir: Yine 0'dan 7'ye, sonra tekrar 0'dan 7'ye giden numaralar vardır. Pencere boyutu bir eksiktir, örneğin 0'dan 6'ya kadar olan kısım. Alıcı aldığı ve doğruladığı her çerçeveyi penceresinde uygun yere yerleştirir ve işaretçiyi sağa ilerletir.

Alıcı her çerçeve için ayrı bir ACK gönderebileceği gibi birkaçı için tek bir ACK da gönderebilir. Bunda bir sınırlama yoktur. Örneğin 0, 1, 2, 3 alındıysa hepsi tek ACK ile onaylanabilir. ACK'lar yollandıktan sonra pencere yine kaydırılır.

5.4 Örnek Akış

Solda gönderici, sağda alıcı vardır. Başlangıçta iki pencere aynı konumdadır ($n = 3$, numaralar 0–7, pencere 7).

1. Gönderici Data 0'ı yollar. Göndericide işaretçi 1'e geçer. Alıcı Data 0'ı alır, onun işaretçisi de 1'e geçer.
2. Gönderici Data 1'i yollar. Alıcıda işaretçi 2'ye geçer. Alıcı 0 için henüz ACK göndermemişti. Şimdi ikisi için birden **ACK 2** gönderir. Dikkat: Data 1'i aldı ama ACK 2 gönderdi, yani bir sonraki adımda almayı beklediği çerçevenin numarasını.
3. ACK 2 ile iki taraftaki pencereler iki kayar.
4. Data 2 gelir, alıcı **ACK 3** gönderir. Pencereler bir kayar.

Her ACK'dan sonra iki taraftaki pencereler de kayar: önce iki, sonra bir. Yapı bu şekilde devam eder.

5.5 Pencere Neden 2^n Değil de $2^n - 1$?

$n = 3$ olsun ve pencere 8 alınsın.

1. Gönderici 0 numaralı çerçeveyi yollar, ACK 1 gelir. Pencere kayar.
2. Gönderici 1, 2, 3, 4, 5, 6, 7, 0 çerçevelerini, yani 8 çerçeveyi art arda gönderir. Gönderici ACK gelmese bile pencere bitene kadar göndermeye devam eder.
3. Bir süre sonra yeniden bir **ACK 1** çerçevesi gelir.

Bu ACK 1 hangi 1'i kastediyor? İki ihtimal vardır:

- Alıcı 8 çerçevenin hepsini almış ve hepsine tek bir çerçeveyle cevap vermiştir. En son 0 gönderildiği için yeni turdaki 1'i bekliyordur.
- Alıcı hiçbirini almamıştır ve daha önce gönderilmiş olan ACK 1 tekrarlanmıştır (duplicate).

Gönderici bu iki durumu ayırt edemez. Bu bilinmezi çözemeyeceğimiz için pencere boyutu 2^n yerine $2^n - 1$ seçilir. Pencere 7 olsaydı gönderici 1'den 7'ye kadar yollardı. Hepsi alınsaydı ACK 0, hiçbiri alınmasaydı ACK 1 gelirdi ve iki durum ayrılırdı. Python ile kontrol edildi: Pencere 8'de iki durumda da ACK 1 gelir, pencere 7'de ACK 0 ve ACK 1 farklıdır.

> **Ek bilgi (derste anlatılmadı):** $2^n - 1$ sınırı, alıcının yalnız sıradaki çerçeveyi kabul ettiği Go-Back-N için geçerlidir. Alıcının sıra dışı çerçeveleri de tamponladığı Selective Repeat'te pencere en fazla 2^{n-1} olabilir. Aksi halde alıcı, yeniden gönderilen eski bir çerçeveyi yeni turun çerçevesi sanabilir.

5.6 Kayan Pencerede Hat Kullanım Oranı

Kayan pencerede kullanım oranı hesaplanırken yine a parametresinden faydalanılır. Fakat bu kez pencere boyutu W da hesaba girer. Pencere boyutu hattın karakteristiğine uygun seçilmelidir. Hattın karakteristiği, yayılma gecikmesi t_{prop} ve çerçeve iletim süresi t_{frame} ile ifade edilir.

Pencere boyutunun seçiminde genelde şu kural kullanılır:

$$W \geq 2a + 1$$

Bu kuralın nedeni şudur: $W \geq 2a + 1$ seçilirse, herhangi bir çerçeveye ait ACK, pencerenin tamamı boşalmadan göndericiye ulaşır. Gönderici hiç durmadan gönderebilir. Bu, gerçekte tam olmasa da kuramsal olarak hat kullanım oranının %100 olmasını sağlar.

Pencere bu kurala göre seçilmezse gönderici penceresini doldurup bitirir ama alıcının ACK'ları henüz göndericiye ulaşmamış olur. Gönderici beklemek zorunda kalır ve verim düşer. Bu durumda W çerçeve, $2a + 1$ çerçeve süresinde gönderilmiş olur (zaman bir çerçeve süresine göre normalize edilirse, ara adım eklendi: ilk ACK $t_{frame} + 2t_{prop}$ sonra gelir, bu da $1 + 2a$ çerçeve süresidir).

$$U = \begin{cases} 1, & W \geq 2a + 1 \\ \dfrac{W}{2a + 1}, & W < 2a + 1 \end{cases}$$

Ders sonrası not: Hoca $2a + 1$ ifadesini "bant genişliği–gecikme çarpımının (bandwidth-delay) iki katı artı 1" diye anlatıyor. Doğrusu, a'nın yayılma gecikmesinin çerçeve süresi cinsinden ifadesi olduğudur. Yani a, bant genişliği–gecikme çarpımının ($R \cdot t_{prop}$, bit) çerçeve boyuna (L) bölünmüş halidir: $a = R\,t_{prop}/L$.

6. Hata Denetimi ve ARQ

6.1 Neden Gerekli?

Hata denetiminde, gönderdiğimiz bilginin karşı tarafta gönderdiğimiz şekilde alınıp alınmadığını anlamamız gerekir. Alıcı bilginin hatalı olduğunu, geçen hafta konuştuğumuz hata algılama ve hata düzeltme kodlarıyla (error detection / error correction codes) anlar: VRC, CRC vb.

Şimdiye kadar konuşulan akış denetimi hep hatasız, mükemmel bir yapıyı varsayıyordu: Çerçeve gönderilir, karşıya ulaşırsa ACK gelir. Hata olduğunda ise göndericiden aynı çerçeveyi yeniden göndermesi istenir. Buna otomatik yeniden gönderim isteği (ARQ, Automatic Repeat reQuest) denir: Bir çerçeve gönderildi, karşı taraf ACK yerine NAK gönderdi, gönderici aynı çerçeveyi yeniden gönderir.

6.2 Dur ve Bekle ARQ

Dur ve bekle yapısı zaten basittir: Bir çerçeve gönderilir, cevap gelince yeni çerçeve gönderilir. Buna yalnız hata durumları eklenir. Gönderici her çerçeve için bir alındı bilgisi bekler ve alındı bilgisi gelince çerçeveyi tamponundan düşürür. Artık geri bildirimin olumlu olup olmadığını da kontrol eder:

1. Gönderici veri çerçevesini yollar, ACK alır.
2. Bir sonraki veriyi yollar, bu kez NAK alır.
3. Aynı veriyi yeniden yollar.

Ayrıca hem gönderilen veri hem de ona karşılık üretilen ACK ya da NAK hatta kaybolabilir. Gönderici hiçbir cevap alamazsa hangisinin başına geldiğini bilemez. Yapılabilecek tek şey bilginin yeniden gönderilmesidir.

6.3 Kayan Pencerede ARQ

Kayan pencerede hata olduğunda iki temel yöntem vardır:

- **N geri dön (Go-Back-N):** Bir noktada hata alındığında hatalı noktaya dönülür ve oradan itibaren yeniden gönderilir. Hoca "dönüp" kelimesinden çağrışım kurmanın kolay olduğunu söyledi.
- **Seçici ret (Selective Reject) ya da seçici yineleme (Selective Repeat):** Kaynaklarda iki ad da geçer, Selective Repeat daha yaygın adıdır. Göndericiye yalnız hatalı çerçeve söylenir. Gönderici yalnız o çerçeveyi yeniden hatta çıkarır.

İkisinin de artıları ve eksileri vardır.

Kayan pencerede ARQ'da ACK, bir sonraki gönderilmesi beklenen çerçeveyi gösterir. NAK ise hatalı olan çerçeveyi gösterir. NAK'tan sonra göndericinin nasıl davrandığı yöntemin adını belirler. Örnek: 0, 1, 2, 3, 4 gönderildi ve 2 numaralı çerçeve için NAK geldi.

- Gönderici 2'ye dönüp 2, 3, 4, 5, 6'yı gönderiyorsa bu Go-Back-N'dir: Hatalı çerçeveden itibaren her şey yeniden gönderilir.
- Selective Repeat'te NAK 2 geldiğinde gönderici diyelim 3, 4 ve 5'i göndermişti. 6'yı göndermek yerine önce 2'yi gönderir, sonra 6 ile devam eder.

  Ders sonrası not: Hoca bu yeri "önce 2'yi gönderiyorsunuz, sonra 2'yi gönderiyorsunuz" diye söylüyor. Bağlama göre doğrusu "önce 2'yi, sonra 6'yı" olmalıdır.

6.4 RR, REJ ve Zamanlayıcı

Kayan pencerede ARQ'da ACK, alıcı hazır (RR, Receive Ready) bilgisiyle gönderilir. Örneğin alıcıdan art arda RR 3 ve RR 6 geldi:

- RR 3: “0, 1, 2’yi aldım, 3’ü bekliyorum.”
- RR 6: “3, 4, 5 de geldi, 6’yı bekliyorum.”

Olumsuz bildirim Go-Back-N’de REJ (reject), Selective Repeat’te SREJ (selective reject) ile ifade edilir.

Kayan pencerede ARQ yapılıyorsa göndericide bir zamanlayıcı (timer) gerekir. Amaç, gönderilen çerçevenin yerine varıp varmadığını anlamaktır. Zamanlayıcı dolduğunda, gönderilen veri gitmediği ya da karşılığında üretilen ACK geri gelmediği için bilgi yeniden gönderilir.

7. Go-Back-N

7.1 Çalışma Şekli

Go-Back-N’de çerçeveler pencere içinde sırayla gönderilir. Hatalı bir çerçeve olursa onunla ilgili bir geri dönüş beklenir. Go-Back-N’in çalışma yöntemi daha basittir ve gerçeklemesi daha kolaydır.

Örnek (slayttan):

1. Gönderici Data 0, 1, 2, 3 ve 4’ü gönderir.
2. 0, 1 ve 2 gönderildiği sırada alıcı **RR 3** der: “3. çerçeveyi almaya hazırım.”
3. Gönderici 3’ü gönderir, ama 3 karşıya hatalı ulaşır.
4. Alıcı **REJ 3** gönderir: “3’ü yeniden gönder.”
5. Gönderici o sırada 5. çerçevedeyse onları bırakır ve 3. çerçeveden itibaren her şeyi yeniden gönderir.

Bunun temel nedeni şudur: Alıcı tarafında 3. çerçevede hata olduğu anlaşıldıktan sonra gelen çerçeveler hiçbir şekilde değerlendirilmez.

Soru: Hazır çerçeve gelmiş, neden değerlendirmiyorsun?

Cevap: Gerçekleme kolaylığı. Bu yapıda ek bir tamponlama, çerçeveleri sıralama, yeni bir zamanlayıcı gibi hiçbir şeye ihtiyaç yoktur. Arada gelen çerçeveleri tamponlayan, sıralayan, düzenleyen, akışta atlamalara izin veren bir yapı kurulmaya karar verilirse, onun adı zaten Selective Repeat ya da Selective Reject olur. O zaman daha karmaşık bir yapı kurmak gerekir. İki yöntem arasındaki temel fark budur.

Hoca bunu donanım tarafına bağladı: Veri bağlantı katmanının bir kısmı ağ cihazının üzerinde, örneğin Ethernet kartında, bir kısmı da işletim sistemi içinde gerçeklenir. Tampon kullanmak, tampon için zamanlayıcı kullanmak gibi şeyler kartın maliyetini artırır. Bu yüzden çok tercih edilmez. Basit olan daha çok tercih edilir.

7.2 Kayıp Çerçeve

Go-Back-N daha basit bir yapıdır ve çerçeveleri sırayla almayı bekler. Alıcı 0 ve 1’den sonra 2 yerine 3’ü görürse 2’nin kaybolduğunu ya da kendisine ulaşmadığını anlar ve **REJ 2** gönderir. Gönderici yeniden 2’den başlayarak gönderir.

7.3 Hiç Cevap Gelmezse: Poll Biti

Veri kaybı farklı şekillerde ortaya çıkabilir. Hata illa veri çerçevelerinde olmak zorunda değildir. ACK çerçevelerinde de hata ya da kayıp olabilir.

Örneğin Go-Back-N kullanıyorsunuz. Bütün çerçevelerinizi gönderdiniz, pencere bitti, ama size çerçevelerin alındığına dair hiçbir bilgi gelmedi. O zaman son çerçeveyi gönderdikten sonra belli bir süre beklenir ve zaman aşımı olur. Ardından karşı tarafa P (poll) biti set edilmiş bir çerçeve gönderilir. Bununla alıcıya ne durumda olduğu sorulur. Alıcı bütün çerçeveleri almışsa “bir sonraki beklediğim çerçeve şu” der. O zaman bir sıkıntı yoktur ve devam edilir.

8. Sırtlama (Piggybacking)

Şimdiye kadar hep veri çerçevesi ve ACK çerçevesi ayrı düşünüldü. İletim tek yönlüyse, yani veri yalnız bir noktadan diğerine aktarılıyorsa, içinde veri olmayan, yalnız verinin alındığını

bildiren ayrı bir ACK çerçevesi kullanılması gerekir.

Fakat iki yönlü haberleşme varsa hat daha etkin kullanılabilir. A'dan B'ye veri çerçeveleri giderken B'den A'ya da veri çerçeveleri gidiyorsa, B göndereceği ACK bilgisini kendi veri çerçevelerinin içine ekleyebilir. Çerçevede bunun için bir alan vardır. Buna sırtlama (piggybacking) denir. ACK bilgisi veri çerçevesinin içine gömülür ve hat kullanım oranı artar.

Bu yüzden çerçevelere cevap beklerken zaman aşımı süreleri biraz daha uzun tutulabilir. Gönderici çerçeveyi gönderdi, karşı taraf aldı, ama o an gönderecek bir verisi olmayabilir. Örneğin 100 milisaniye kadar fazladan beklemek, gönderilecek bir verinin oluşmasına, o katmana verilmesine ve ACK'nın onun içine eklenmesine imkân verebilir. Hat kullanım oranını artırmak için bu bekleme biraz uzatılabilir.

9. Seçici Ret / Seçici Yineleme (Selective Reject / Selective Repeat)

Bu yöntemde de hata olduğunda bu, SREJ ile belirtilir. Yani hangi çerçevede hata olduğu söylenir. Bu aynı zamanda ondan önceki çerçevelerin hatasız alındığı anlamına gelir.

Örnek 1 (sözlü): Gönderici 0, 1, 2, 3'ü gönderdi ve SREJ 2 aldı. Bu, 0 ve 1'in düzgün alındığı, 2'de hata olduğu anlamına gelir. 3 ile ilgili henüz bir geri dönüş olmamış olabilir.

- Gönderici 3'ü tamponunda tutmaya devam eder. 2 de zaten tamponundadır.
- 2'nin hatalı olduğunu bildiği için 2'yi yeniden gönderir.
- En son 3'ü göndermişti. Şimdi 2'yi gönderir ve 4'ten devam eder.

Yani çerçeveler artık sırayla değil, sıra dışı (out of order) gönderilir.

Örnek 2 (slayttan): 0, 1, 2, 3, 4 gönderilir. 2'nin hatalı olduğu bilgisi gelir. 0 ve 1 zaten doğru alınmıştır. 2 yeniden gönderilir. Gönderici 4'te kalmıştı, 5, 6 diye devam eder.

Bu, alıcı tarafında da bir yük getirir. Çerçeveler sıra dışı gelebildiği için alıcı onları sıraya koymalıdır. Eksik varsa eksiği fark etmelidir. Gelmeyen bir çerçeve de olabilir. Örneğin 0, 1, 2, 3, 4 gönderildi, alıcı 0, 2 ve 3'ü görüyor, aradaki 1 yok. Demek ki bir sorun olmuştur. Alıcı bununla ilgili geri dönüş yapmalı ve sıra bozukluğunu bildirmelidir ki çerçeve yeniden gönderilebilsin.

Dur ve bekle ile konuşulan durum burada da olasıdır. Çerçeveler gönderildi ama hiçbir şey gelmiyor: ne RR, ne olumsuz bir bildirim. O zaman gönderici alıcıya bir sorgu göndererek durumu sorar. Böylece çerçevenin gerçekten gidip gitmediği anlaşılır.

Benzer bir senaryo: Gönderici 0, 1, 2, 3, 4, 5, 6 gönderdi. 3. çerçeveden sonra alıcı aslında RR 3 gönderdi, ama bu göndericiye ulaşmadı. Gönderici bütün pencereyi boşalttı, artık bir cevap gelmiş olması gerekiyor, ama hiçbir şey gelmedi. Yine yoklama (polling) yapılır ve karşı tarafın durumu sorulur. Durum normalse kalınan yerden devam edilir.

Örnek Sorular

Soru 1 (derste sözlü, 00:39:14)

İki nokta arası mesafe 1000 km'dir. Veri hızı 155,52 Mbps, yayılma hızı 2×10^8 m/s, çerçeve boyu 424 bittir. Dur ve bekle yöntemi kullanılırsa hattın kullanım oranı ne olur?

Hoca hesaplamak isteyen olup olmadığını sordu, sonra çözüm yolunu gösterdi.

- $t_{prop} = d/v = 10^6/(2 \times 10^8) = 5$ ms
- $t_{frame} = L/R = 424/(155{,}52 \times 10^6) \approx 2{,}726\ \mu$s
- $a = t_{prop}/t_{frame} \approx 1834$
- $U = 1/(1+2a) \approx 1/3669 \approx 2{,}73 \times 10^{-4}$

Sayılar Python ile doğrulandı. Ayrıntılı çözüm Bölüm 4.4'te.

Soru 2 (derste sözlü, 01:30:13)

Ne zaman Selective Repeat, ne zaman Go-Back-N kullanırsınız? Hoca bunu, hata algılama ve hata düzeltme için sorduğu soruya benzetti. Cevabı farklı parametreleri birlikte düşünerek, derinlemesine vermenizi istedi. Derste cevap beklemedi, düşünme sorusu olarak bıraktı.

Çözüm (derste verilmedi): Derste anlatılanlardan şu çerçeve kurulabilir. Go-Back-N alıcıda ek tampon, sıralama ve ek zamanlayıcı istemez. Bu yüzden ucuz ve basittir, ama bir hata olduğunda hatalı çerçeveden sonraki bütün çerçeveler yeniden gönderilir. Bu kayıp, hata oranı düşük ve a küçük (kısa, hızlı) hatlarda azdır. Hata oranı yüksek ve a büyük (uzun mesafeli, yüksek gecikmeli, pencere büyük) hatlarda ise her hata çok sayıda çerçevenin boşa yeniden gönderilmesi demektir. Burada Selective Repeat'in ek tampon ve karmaşıklık maliyeti büyük olasılıkla kendini öder. Alıcı cihazın maliyeti ve bellek kısıtı da kararda rol oynar.

Dersin Toparlaması

Bu derste veri bağlantı katmanının iki temel görevi ele alındı: bağlantı yönetimi ve akış denetimi. Bağlantı yönetiminde noktadan noktaya hatlar için eşit iki uç arasında çalışan ENQ/ACK, çok noktalı hatlar için kontrolün tamamen birincil istasyonda olduğu Poll/Select görüldü. Akış denetiminde dur ve bekle yönteminin basit ama uzun mesafelerde çok verimsiz olduğu, $U = 1/(1+2a)$ formülü ve 1000 km'lik örnekle gösterildi. Kayan pencere bu sorunu pencere içindeki çerçeveleri art arda göndererek çözer. Pencerenin $2^n - 1$ seçilmesi ACK belirsizliğini önler, $W \geq 2a + 1$ seçilmesi ise hattın kuramsal olarak tam kullanılmasını sağlar.

Ardından akış denetimine hata durumları eklenerek ARQ konuşuldu. Dur ve bekle ARQ, Go-Back-N ve Selective Repeat karşılaştırıldı. Go-Back-N'in gerçekleme ve donanım maliyeti açısından basit olduğu, Selective Repeat'in ise alıcıda tamponlama ve sıralama gerektirdiği vurgulandı. İki yönlü haberleşmede ACK'nın veri çerçevesine gömülmesi (piggybacking) de anlatıldı. Hoca derse, hangi durumda hangi ARQ yönteminin seçileceğini düşünme sorusu olarak bırakarak son verdi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.